Parul® University

MEA(R)N Stack Web Development (303105385)

Unit 3 : Node.js & Express.js

Prof. Snehal Thakor

Assistant Professor

Computer Science & Engineering

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Syllabus

- Introduction to Node.js, architecture, and its event-driven model.
- Node.js modules and package management using npm.
- Introduction to Express.js and setting up a basic Express server.
- Middleware and routing in Express.js.
- RESTful API development with Express.js.
- Authentication and security with Passport.js.
- Error handling and logging in Node.js applications.
- Working with third-party APIs using Express.js.
- Deploying Express.js applications to a live server.
- Real-world application: Building a REST API using Node.js and Express.js.

Why Node.js?

➢ What is Node.js?

- Node.js is an open source server environment
- Node.js is free
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server

Introduction

- What is Node.js?

Node.js uses asynchronous programming!

Here is how PHP or ASP handles a file request:

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request.

Here is how Node.js handles a file request:

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.

What can Node.js Do?

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database

➢ What is a Node.js File?

- Node.js files contain tasks that will be executed on certain events
- A typical event is someone trying to access a port on the server
- Node.js files must be initiated on the server before having any effect
- Node.js files have extension ".js"

Download Node.js

The official Node.js website has installation instructions for Node.js: https://nodejs.org

Example:

myfirst.js

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

Save the file on your computer: C:\Users\Your Name\myfirst.js

How to run the Node.js in Command Line Interface

Node.js files must be initiated in the "Command Line Interface" program of your computer.
Navigate to the folder that contains the file "myfirst.js"

Initiate the Node.js File

C:\Users\yourname>node myfirst.js

Now, your computer works as a server!
If anyone tries to access your computer on port 8080, they will get a "Hello World!" message in return!
Start your internet browser, and type in the address: http://localhost:8080

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Node.js Architecture & Event Driven Model

- Node.js Web Application Architecture

Node.js is a JavaScript-based platform mainly used to create I/O-intensive web applications such as chat apps, multimedia streaming sites, etc. It is built on Google Chrome's V8 JavaScript engine.

A web application is software that runs on a server and is rendered by a client browser that accesses all of the application's resources through the Internet.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Node.js Architecture & Event Driven Model

- A typical web application consists of the following components:

• Client: A client refers to the user interacting with the server by sending out requests.
• Server: The server is responsible for receiving client requests, performing appropriate tasks, and returning results to the clients. It bridges the front end and the stored data, allowing clients to perform operations on the data.
• Database: A database is where a web application's data is stored. The data can be created, modified, and deleted depending on the client's request.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

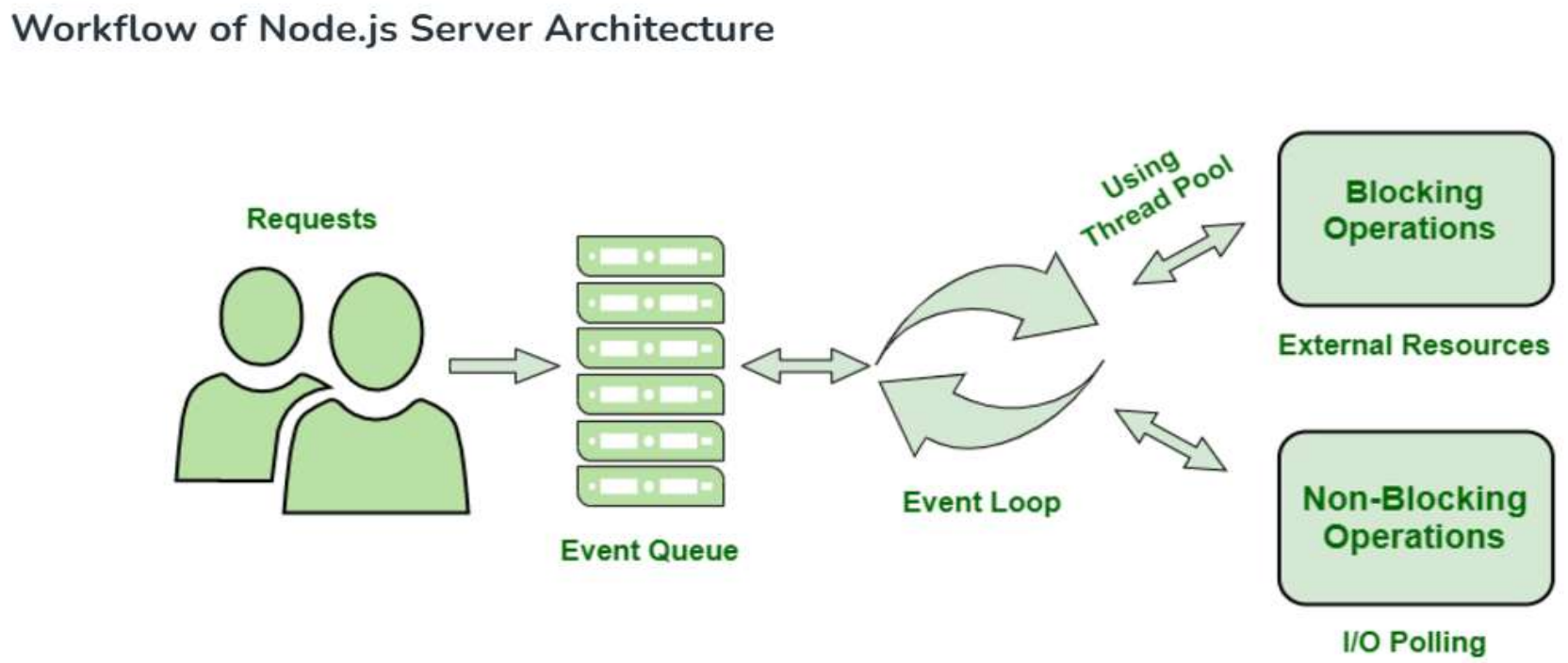
Node.js Event Architecture

To manage several concurrent clients, Node.js employs a “Single Threaded Event Loop” design. The JavaScript event-based model and the JavaScript callback mechanism are employed in the Node.js Processing Model.
It employs two fundamental concepts:

1.Asynchronous model
2.Non-blocking of I/O operations

These features enhance the scalability, performance, and throughput of Node.js web applications.

Workflow of Node.js Server Architecture



Components of Node.js Architecture

•Requests: Depending on the actions that a user needs to perform, the requests to the server can be either blocking (complex) or non-blocking (simple).
•Node.js Server: The Node.js server accepts user requests, processes them, and returns results to the users.
•Event Queue: The main use of Event Queue is to store the incoming client requests and pass them sequentially to the Event Loop.
•Thread Pool: The Thread pool in a Node.js server contains the threads that are available for performing operations required to process requests.
•Event Loop: Event Loop receives requests from the Event Queue and sends out the responses to the clients.
•External Resources: In order to handle blocking client requests, external resources are used. They can be of any type (computation, storage, etc).

Advantages of Node.js Server Architecture

- The Node.js server can efficiently handle a high number of requests by employing the use of Event Queue and Thread Pool.
- There is no need to establish multiple threads because Event Loop processes all requests one at a time, therefore a single thread is sufficient.
- The entire process of serving requests to a Node.js server consumes less memory and server resources since the requests are handled one at a time.

Disadvantages of Node.js Server Architecture

1.Single-Threaded: Limited to one thread; can be a bottleneck for CPU-intensive tasks.
2.Callback Hell: Complex nesting of callbacks can lead to hard-to-maintain code.
3.Performance Bottlenecks: Not optimal for heavy computational tasks due to non-blocking I/O model.
4.Dependency on Outside Libraries: Heavy reliance on third-party libraries can impact stability and security.
5.Inconsistent API: Frequent API changes can lead to backward compatibility issues.
6.Lack of Strong Typing: JavaScript's lack of strong typing can lead to runtime errors and bugs.

Node.js Modules

What is a Module in Node.js?

Consider modules to be the same as JavaScript libraries.

A set of functions you want to include in your application.

- Built-in Modules

Node.js has a set of built-in modules which you can use without any further installation.

- Include Modules

To include a module, use the require() function with the name of the module.

```
var http = require('http');
```

Node.js Modules

Now your application has access to the HTTP module, and is able to create a server:

```
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

Create your own Modules

You can create your own modules, and easily include them in your applications.
The following example creates a module that returns a date and time object

Example

Create a module that returns the current date and time:

```
exports.myDateTime = function () {
  return Date();
};
```

Use the exports keyword to make properties and methods available outside the module file.
Save the code above in a file called "myfirstmodule.js"

Include Your Own Module

Now you can include and use the module in any of your Node.js files.

Example:

Use the module "myfirstmodule" in a Node.js file:

```
var http = require('http');
var dt = require('./myfirstmodule');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write("The date and time are currently: " + dt.myDateTime());
  res.end();
}).listen(8080);
```

Save the code above in a file called "demo_module.js", and initiate the file.

C:\Users\Your Name>node demo_module.js

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Node.js NPM

- What is NPM?

NPM is a package manager for Node.js packages, or modules if you like. www.npmjs.com hosts thousands of free packages to download and use. The NPM program is installed on your computer when you install Node.js

- What is a Package?

A package in Node.js contains all the files you need for a module. Modules are JavaScript libraries you can include in your project.

Downloading & Using Package

- Downloading a package

```
C:\Users\Your Name>npm install upper-case
```

Now you have downloaded and installed your first package!

NPM creates a folder named "node_modules", where the package will be placed. All packages you install in the future will be placed in this folder.

My project now has a folder structure like this:

```
C:\Users\My Name\node_modules\upper-case
```

Downloading & Using Package

Once the package is installed, it is ready to use.
Include the "upper-case" package the same way you include any other module:

```
var uc = require('upper-case’);
```

Create a Node.js file that will convert the output "Hello World!" into upper-case letters. Save the file and run it.

Example:

```
var http = require('http');
var uc = require('upper-case');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(uc.upperCase("Hello World!"));
  res.end();
}).listen(8080);
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Introduction to Express.js and setting up basic Server

- **Express.js** is a fast, flexible and minimalist web framework for Node.js. It's effectively a tool that simplifies building web applications and APIs using JavaScript on the server side. Express is an open-source that is developed and maintained by the Node.js foundation.
- Express.js offers a robust set of features that enhance your productivity and streamline your web application. It makes it easier to organize your application's functionality with middleware and routing. It adds helpful utilities to Node HTTP objects and facilitates the rendering of dynamic HTTP objects.
- With Express, you can easily handle routes, requests, and responses, which makes the process of creating robust and scalable applications much easier. Moreover, it is a lightweight and flexible framework that is easy to learn and comes loaded with middleware options.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Express Installation

Install Express using npm:

```
npm install express
```

Basic Example of an Express App:

```
const express = require('express');
const app = express();

// Define routes and middleware here
// ...

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Explanation:

Import the '**express**' module to create a web application using Node.js.
Initialize an Express app using const app = express();.
Add routes (endpoints) and middleware functions to handle requests and perform tasks like authentication or logging.
Specify a port (defaulting to 3000) for the server to listen on.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Middleware in Express.js

What is Middleware in Express JS?

Middleware is a request handler that allows you to intercept and manipulate requests and responses before they reach route handlers. They are the functions that are invoked by the Express.js routing layer.

It is a flexible tool that helps add functionalities like logging, authentication, error handling, and more to Express applications.

Syntax

The basic syntax for the middleware functions is

app.get(path, (req, res, next) => {}, (req, res) => {})

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Middleware in Express.js

- Middleware functions take 3 arguments: the **request object**, the **response object**, and the **next function** in the application's request-response cycle, i.e., two objects and one function.

• Middleware functions execute some code that can have side effects on the app and usually add information to the request or response objects. They are also capable of ending the cycle by sending a response when some condition is satisfied. If they don't send the response when they are done, they start executing the **next function** in the stack. This triggers calling the 3rd argument, next().

•The middle part **(req, res, next)=>{}** is the middleware function. Here we generally perform the required actions before the user can view the webpage or call the data and many other functions. So let us create our middleware and see its uses.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Middleware in Express.js

Types of Middleware

Express JS offers different types of middleware and you should choose the middleware on the basis of functionality required.

- **Application-level middleware:** Bound to the entire application using app.use() or app.METHOD() and executes for all routes.
- **Router-level middleware**: Associated with specific routes using router.use() or router.METHOD() and executes for routes defined within that router.
- **Error-handling middleware:** Handles errors during the request-response cycle. Defined with four parameters (err, req, res, next).
- **Built-in middleware:** Provided by Express (e.g., express.static, express.json, etc.).
- **Third-party middleware**: Developed by external packages (e.g., body-parser, morgan, etc.).

Middleware in Express.js

Steps to Implement Middleware in Express

- Step 1: Initialize the node in the project using the following command.

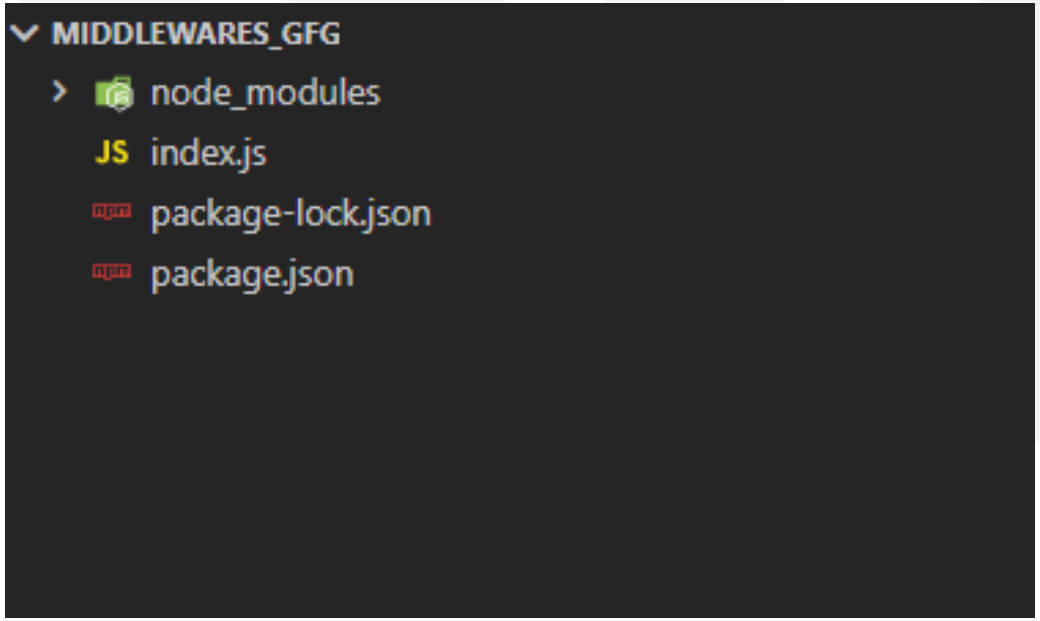
  npm init -y
- Step 2: Install the required dependencies.

  npm install express nodemon
- Step 3: In scripts section of package.json file, add the following line.

  "start": "nodemon index.js",

Middleware in Express.js

Project Structure



Middleware in Express.js

- **Set up our express app and send a response to our server.**

```
// Filename - index.js
const express = require("express");
const app = express();
const port = process.env.port || 3000;
app.get("/", (req, res) => {
    res.send(`<div>
    <h2>Welcome to Middleware in Express.js</h2>
    <h5>Tutorial on Middleware</h5>
  </div>`);
});
app.listen(port, () => {
    console.log(`Listening to port ${port}`);
});
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

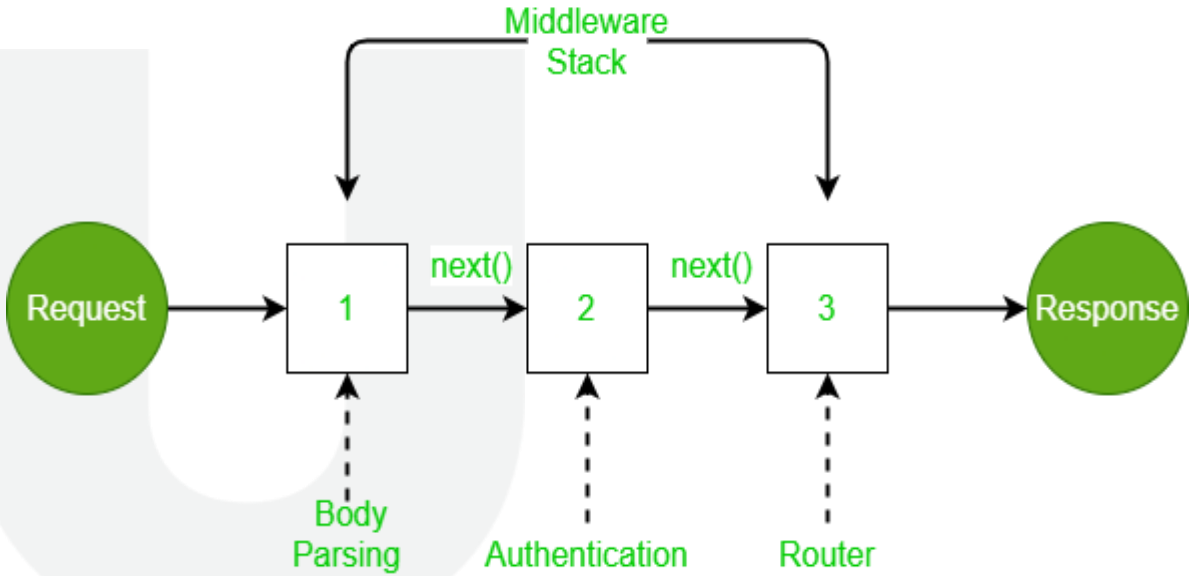
Middleware in Express.js

- Start the application using the following command.

- npm start

Middleware Chaining

- Middleware can be chained from one to another, Hence creating a chain of functions that are executed in order. The last function sends the response back to the browser. So, before sending the response back to the browser the different middleware processes the request.
- The next() function in the express is responsible for calling the next middleware function if there is one.
- Modified requests will be available to each middleware via the next function



Parul® University NAAC A++ ACCREDITED UNIVERSITY

Middleware in Express.js

Advantages of using Middleware

- Middleware can process request objects multiple times before the server works for that request.
- Middleware can be used to add logging and authentication functionality.
- Middleware improves client-side rendering performance.
- Middleware is used for setting some specific HTTP headers.
- Middleware helps with Optimization and better performance.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Routing in Express.js

Routing refers to determining how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, and so on). Each route can have one or more handler functions, which are executed when the route is matched.
Route definition takes the following structure:
app.METHOD(PATH, HANDLER)

Where:

app is an instance of express.
METHOD is an HTTP request method, in lowercase.
PATH is a path on the server.
HANDLER is the function executed when the route is matched.

Routing in Express.js

Examples illustrate defining simple routes.

- Respond with Hello World! on the homepage: **app.get('/', (req, res) => {**
  res.send('Hello World!')
  })
- Respond to POST request on the root route (/), the application's home page:

```
app.post('/', (req, res) => {
  res.send('Got a POST request')
})
```

- Respond to a PUT request to the /user route:

```
app.put('/user', (req, res) => {
  res.send('Got a PUT request at /user')
})
```

- Respond to a DELETE request to the /user route:

```
app.delete('/user', (req, res) => {
  res.send('Got a DELETE request at /user')
})
```

Routing in Express.js

Route Parameters

Definition: Route parameters allow you to capture values from the URL.

Syntax: `app.METHOD('/path/:parameter', handler)`

- Example:

```js
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID: ${userId}`);
});
```

Output**: `GET /user/123` → `User ID: 123`

- "Query Parameters"
- - Definition: Query parameters are passed as key-value pairs in the URL after a `?`.
- - **Syntax**: `/path?key=value&key2=value2`
- - **Accessing Query Parameters**: `req.query`

Routing in Express.js

- **Example**:

```
js
app.get('/search', (req, res) => {
  const query = req.query.q;
  res.send(`Search results for: ${query}`);
});
```

- **Output**: `GET /search?q=express` → `Search results for: express`

Routing in Express.js

Route Handlers and Middleware
- **Middleware**: Functions that process requests before they reach the route handler.
- **Example**:
  ```js
  const authenticate = (req, res, next) => {
    if (req.isAuthenticated()) {
      next(); // Proceed to the next middleware or route handler
    } else {
      res.status(403).send('Forbidden');
    }
  };

  app.get('/profile', authenticate, (req, res) => {
    res.send('Profile page');
  });
  ```

Routing in Express.js

***Route Chaining**
- **Definition**: You can chain multiple route handlers for the same path.
- **Syntax**:
 ```
 js
 app.route('/book')
   .get((req, res) => res.send('GET request'))
   .post((req, res) => res.send('POST request'));
 ```

 Route Groups (Express Router)
- **Definition**: Use `express.Router()` to create modular route handlers.
- **Example**:
 ```
 js
 const router = express.Router();
   router.get('/', (req, res) => {
   res.send('Home page');
 });  app.use('/home', router);
 ```

Routing in Express.js

Handling Dynamic Routes
- **Dynamic Segments**: Use placeholders in the URL path to create dynamic routes.
- **Example**:
 ```js
 app.get('/products/:productId', (req, res) => {
  const productId = req.params.productId;
  res.send(`Product ID: ${productId}`);
 });
404 and Error Handling**
- **404 Error**: Handling invalid routes.
 ```js
 app.use((req, res, next) => {
  res.status(404).send('Page Not Found');
 });
 ```
- **Error Handling Middleware**:
 ```js
 app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).send('Something went wrong');
 });

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

# RESTful API development in Express.js

- ***Introduction to RESTful APIs***
- ***Why Express.js for API Development?***
- ***Setting up an Express.js Project***
- ***Key Concepts of RESTful APIs***

➢ ***Introduction to RESTful APIs***

- *What is a RESTful API?*
- *Representational State Transfer (REST) is an architectural style.*
- *RESTful APIs are web services that adhere to REST principles.*
- *Core Principles:*
- *Stateless communication*
- *Client-server architecture*
- *Uniform interface*
- *Resource-based (using URIs to identify resources)*
- *HTTP methods to perform CRUD operations*

Parul® University NAAC A++ ACCREDITED UNIVERSITY


# RESTful API development in Express.js

**Why Use Express.js for API Development?**

**Express.js Overview:**

- Minimalist web framework for Node.js.
- Simplifies routing, middleware, and request handling.

**Advantages:**

- Fast, lightweight, and flexible.
- Easy to set up and scale.
- Large ecosystem of middleware.
- Strong community support.

**When to Use:**

- Developing REST APIs for web/mobile apps.
- Handling large amounts of HTTP requests efficiently.


# RESTful API development in Express.js

- Setting Up an Express.js Project

**Step 1:** Initialize a new Node.js project

```
npm init –y
```

Step 2: Install Express.js

```
npm install express
```

Step 3: Create a basic server (e.g.,app.js)

```
const express = require('express');
const app = express();
const port = 3000;
    app.get('/', (req, res) => {
      res.send('Hello World');
    });
    app.listen(port, () => {
      console.log(`Server running on http://localhost:${port}`);
    });
```

RESTful API development in Express.js

- Setting Up an Express.js Project

Step 1: Initialize a new Node.js project

```
npm init –y
```

Step 2: Install Express.js

```
npm install express
```

Step 3: Create a basic server (e.g.,app.js)

```
const express = require('express');
const app = express();
const port = 3000;
    app.get('/', (req, res) => {
      res.send('Hello World');
    });
    app.listen(port, () => {
      console.log(`Server running on http://localhost:${port}`);
    });
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Authentication & Security with Passport.js

- **Definition:** Passport.js is a lightweight and flexible authentication middleware for Node.js.

  Purpose: It simplifies user authentication by providing easy integration with various strategies (e.g., local, OAuth, OpenID, SSO).

  Key Features:

  Simple API

  Support for multiple authentication strategies

  Can be used with any Express.js-based application

- **Passport.js Overview**

How it works:

- **Passport.js does not handle sessions, cookies, or databases; it focuses only on authentication.**
- **It works by attaching an authentication strategy (like Google or Facebook) to an Express route.**

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Authentication & Security with Passport.js

- **Key Components of Passport.js**
  - **Strategies: Pre-configured methods of authentication (e.g., Local, JWT, OAuth).**
  - **Serialization & Deserialization:**
    - **Serialization: How the user information is stored in the session.**
    - **Deserialization: How user info is retrieved from the session.**
  - **Middleware: passport.authenticate() is used for authentication in routes.**
  - **Session Management: Optional use of sessions to persist login state.**

- **Types of Authentication Strategies**
  - **Local Strategy: Username and password authentication.**
  - **OAuth 2.0: Integration with third-party services (Google, Facebook, GitHub).**
  - **JWT Authentication: JSON Web Tokens for stateless authentication.**
  - **OpenID Connect: For single sign-on (SSO) authentication.**
  - **Custom Strategies: For custom authentication needs.**

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Authentication & Security with Passport.js

- **Key Components of Passport.js**
  - **Strategies: Pre-configured methods of authentication (e.g., Local, JWT, OAuth).**
  - **Serialization & Deserialization:**
    - **Serialization: How the user information is stored in the session.**
    - **Deserialization: How user info is retrieved from the session.**
  - **Middleware: passport.authenticate() is used for authentication in routes.**
  - **Session Management: Optional use of sessions to persist login state.**

- **Types of Authentication Strategies**
  - **Local Strategy: Username and password authentication.**
  - **OAuth 2.0: Integration with third-party services (Google, Facebook, GitHub).**
  - **JWT Authentication: JSON Web Tokens for stateless authentication.**
  - **OpenID Connect: For single sign-on (SSO) authentication.**
  - **Custom Strategies: For custom authentication needs.**

Authentication & Security with Passport.js

```
passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username: username }, function(err, user) {
      if (err) return done(err);
      if (!user) return done(null, false, { message: 'Incorrect username.' });
      if (!bcrypt.compareSync(password, user.password)) {
        return done(null, false, { message: 'Incorrect password.' });
      }
      return done(null, user);
    });
  }
));
```

Explanation: This example shows how Passport authenticates using the LocalStrategy with a username and password check.

Authentication & Security with Passport.js

```
passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username: username }, function(err, user) {
      if (err) return done(err);
      if (!user) return done(null, false, { message: 'Incorrect username.' });
      if (!bcrypt.compareSync(password, user.password)) {
        return done(null, false, { message: 'Incorrect password.' });
      }
      return done(null, user);
    });
  }
));
```

Explanation: This example shows how Passport authenticates using the LocalStrategy with a username and password check.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Authentication & Security with Passport.js

Benefits of Using Passport.js

- **Flexibility: Supports over 500 authentication strategies.**
- **Ease of Use: Simple integration with Express apps.**
- **Extensibility: Custom strategies can be created.**
- **Security: Supports industry-standard security protocols (OAuth, JWT).**
- **Scalability: Easily scales with complex authentication requirements.**

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Authentication & Security with Passport.js

- **Benefits of Using Passport.js**
- **Flexibility: Supports over 500 authentication strategies.**
- **Ease of Use: Simple integration with Express apps.**
- **Extensibility: Custom strategies can be created.**
- **Security: Supports industry-standard security protocols (OAuth, JWT).**
- **Scalability: Easily scales with complex authentication requirements.**
- **Best Practices for Authentication Security**
- **Password Hashing:** Use bcrypt or argon2 for hashing passwords.
- **JWT Expiry:** Ensure JWTs expire after a reasonable time and refresh tokens are used.
- **Session Storage:** Use secure, server-side storage for sessions (e.g., Redis).
- **Two-Factor Authentication (2FA):** Implement 2FA for added security.
- **Regular Security Audits:** Monitor and patch security vulnerabilities regularly.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Error handling and logging in Node.js application

➢ Importance of Error Handling

Purpose:

Prevent application crashes.

Improve user experience by managing errors gracefully.

Ensure security by not revealing sensitive information.

Simplify debugging and maintenance.

➢ Types of Errors in Node.js

- **Synchronous Errors:** Occur during regular function calls (e.g., parsing, computation).
- **Asynchronous Errors:** Occur in async code (e.g., callbacks, promises).
- **System Errors:** Low-level system issues (e.g., file system, network).
- **Application Errors:** Custom application-specific errors (e.g., invalid input).

Error handling and logging in Node.js application

➢ **Error Handling in Synchronous Code**

- **Using try-catch blocks:**

```
try {
  const data = JSON.parse(jsonString); // May throw error
} catch (error) {
  console.error("Error:", error);
}
```

➢ **Error Handling in Asynchronous Code**

- **Callbacks:** Use the error-first pattern.

```
fs.readFile('file.txt', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  console.log(data);
});
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Error handling and logging in Node.js application

- **Promises: Use .catch() for handling errors.**

```
someAsyncFunction().catch(err => {
  console.error('Error:', err);
});
```

Async/Await: Use try-catch for async functions.

```
async function fetchData() {
  try {
    const data = await someAsyncFunction();
    console.log(data);
  } catch (err) {
    console.error('Error:', err);
  }
}
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Error handling and logging in Node.js application

➢ **Logging in Node.js Applications**

- **Why Logging?**

  Helps track errors, application flow, and performance.
  Simplifies debugging and monitoring.
  Crucial for production applications to diagnose issues.

➢ **Common Logging Libraries**

1.Winston:

Flexible, supports multiple transports (console, file, HTTP).

```
const winston = require('winston');
const logger = winston.createLogger({
  transports: [new winston.transports.Console()]
});
logger.info('App started');
```

Error handling and logging in Node.js application

2.Morgan:

HTTP request logging for Express.js applications

```
const morgan = require('morgan');
app.use(morgan('combined')); // Logs HTTP requests
```

3.Pino: Fast and lightweight for performance-critical applications.

```
const pino = require('pino');
const logger = pino();
logger.info('App started');
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Error handling and logging in Node.js application

- **Best Practices for Logging**

1. **Use Log Levels: (info, warn, error) for categorizing logs.**
2. **Structured Logs: Log in a format like JSON for easier parsing.**
3. **Sensitive Data: Avoid logging sensitive information (e.g., passwords).**
4. **Centralized Logging: Aggregate logs from multiple instances in production (e.g., ELK Stack, Datadog).**

- **Error Handling Best Practices**

1. Always Handle Errors: Use try-catch or proper error-checking in callbacks.
2. Graceful Shutdown: Handle uncaught exceptions and unhandled promise rejections.

3.Custom Errors: Use custom error classes for consistent error handling.

```
class CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = 'CustomError';
  }
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Working with third-party API using Express.js

Understanding Third-Party APIs

Examples:

- **Weather API** (e.g., OpenWeatherMap)
- **Payment API** (e.g., Stripe)
- **Authentication API** (e.g., Google OAuth)

Each API usually requires an **API key** and provides a **REST endpoint** (HTTP methods like GET, POST, etc.).

Making API Calls in Express.js

Use libraries like axios or node-fetch to interact with third-party APIs.

npm install axios

Working with third-party API using Express.js

Example code to fetch data from a third-party API (e.g., Weather API):

```
const axios = require('axios');

app.get('/weather', async (req, res) => {
  try {
    const response = await
axios.get('https://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_API_KEY');
    res.json(response.data);
  } catch (error) {
    res.status(500).send('Error fetching weather data');
  }
});
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Working with third-party API using Express.js

➢ **Handling API Keys Securely**

- **Environment Variables: Never hardcode API keys in your code. Use .env files or environment variables.**
- **Install dotenv package:**

```
npm install dotenv
```

- In .env file:

```
API_KEY=your_api_key_here
```

- In your code:

```
require('dotenv').config();
const apiKey = process.env.API_KEY;
```

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Deploying Express.js applications to live Server

➢ **Introduction**

- **What is Deployment?**

  The process of moving an application from development to a live environment (server).

- **Why Deploy Express.js Apps?**

  Make your application accessible to the public.

  Scale and serve requests in a production environment.

  ➢ **Preparing for Deployment**

  1. Test Locally: Ensure that your Express.js app runs smoothly locally.
  2. Check for Errors: Resolve any bugs or issues before deployment.
  3. Secure Your App: Use environment variables for sensitive information (API keys, database credentials).
  4. Production Settings:

  Set NODE_ENV to production.

  Configure logging, error handling, and security (e.g., helmet, CORS).

Deploying Express.js applications to live Server

- **Choosing a Hosting Platform**

Popular Platforms for Express.js Deployment:

Heroku (easy setup, free tier)
DigitalOcean (flexible VPS hosting)
AWS (EC2) (scalable cloud server)
Render (deployment platform for web services)
Vercel or **Netlify** (for serverless deployments)

Choose based on your app's needs (e.g., scalability, cost, ease of use).

- **Deploying on Heroku**

Create a Heroku Account: Sign up at heroku.com.
Install Heroku CLI: Download and install the Heroku Command Line Interface.
Deploy the App:
Initialize Git:

```
git init
git add .
git commit -m "Initial commit"
```

Deploying Express.js applications to live Server

- Create a Heroku app:

  heroku create

- Deploy your app:

  git push heroku master

Access the App: Once deployed, Heroku provides a live URL for your app.

- **Environment Variables in Production**
- Set Environment Variables on Heroku:

Set up environment variables using the Heroku dashboard or CLI:

heroku config:set KEY=value

- Use dotenv locally:

- In development, use .env files for environment variables.
- Use process.env.KEY in your code to access these values securely.

Deploying Express.js applications to live Server

➢ **Deployment on DigitalOcean (VPS)**

1. **Create a Droplet**: Choose a basic droplet (Ubuntu server).
2. **Set Up the Server**:

SSH into the server:

ssh root@your_droplet_ip

Install Node.js and NPM.

3. Transfer Your App: Use Git or SCP to transfer files to the server.

4. Set Up Nginx or Apache: Configure Nginx as a reverse proxy for your Express app.
5. Start the App:

- Use a process manager like **PM2** to keep the app running:
- npm install pm2 -g
- pm2 start app.js

Make sure your server is accessible through a domain name.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Deploying Express.js applications to live Server

- **Security Considerations**
- **Enable HTTPS**: Use SSL certificates (e.g., via Let's Encrypt) to secure your app.
- **Use Firewalls and Security Tools**: Protect your server and app.
- **Secure Your Database**: Ensure that your database is not publicly accessible and is properly secured.
- **Best Practices for Deployment**
- **Environment Variables**: Never hardcode sensitive data in your code.
- **Performance Optimization**: Minify assets, enable caching, and use a CDN.
- **Backup and Restore**: Ensure regular backups of your database and important data.
- **Regular Updates**: Keep your dependencies and server software up to date.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Real –World application : Building a REST API using Node.js and Express.js

➢ **Building a REST API with Node.js and Express.js**

- verview:

- REST (Representational State Transfer) is an architectural style for designing networked applications, using HTTP requests for CRUD operations.
- Node.js: A JavaScript runtime that allows building scalable server-side applications.
- Express.js: A minimal and flexible Node.js web application framework that simplifies building APIs.

Parul® University NAAC A++ ACCREDITED UNIVERSITY

DIGITAL LEARNING CONTENT

Real –World application : Building a REST API using Node.js and Express.js

➢ **Steps:**

1. **Setup Environment:**
   - **Install Node.js and Express using npm.**
   - **Initialize a new project (npm init).**
2. **Create API Endpoints:**
   - **Define routes (e.g., GET, POST, PUT, DELETE) to handle client requests.**
   - **Use Express.js to route HTTP methods to specific functions.**
3. **Handle Requests:**
   - **Use Express middleware to process requests (e.g., parsing JSON data).**
4. **Send Responses:**
   - **Return appropriate HTTP status codes and data in JSON format.**

Parul® University NAAC A++ ACCREDITED UNIVERSITY

Real –World application : Building a REST API using Node.js and Express.js

- **Benefits:**
  - **Fast, scalable, and easy to maintain.**
  - **Leverages JavaScript for both client and server-side development.**
  - **Rich ecosystem with reusable modules and packages.**

```
const express = require('express');
const app = express();
app.get('/api/hello', (req, res) => {
  res.json({ message: 'Hello, World!' });
});
app.listen(3000, () => console.log('Server running on port 3000'));
```

DIGITAL LEARNING CONTENT

Parul® University

www.paruluniversity.ac.in